



LLM-based Satisfiability Checking of String Requirements by Consistent Data and Checker Generation

Boqi Chen^{*} ECE McGill University Montreal, Canada
 Aren A. Babikian^{*} Dep't of Computer Science University of Toronto Toronto, Canada
 Shuzhao Feng ECE McGill University Montreal, Canada
 Dániel Varró[†] IDA Linköping University Linköping, Sweden
 Gunter Mussbacher ECE McGill University Montreal, Canada

Abstract—Requirements over strings, commonly represented using natural language (NL), are particularly relevant for software systems due to their heavy reliance on string data manipulation. While individual requirements can usually be analyzed manually, verifying properties (e.g., satisfiability) over sets of NL requirements is particularly challenging. Formal approaches (e.g., SMT solvers) may efficiently verify such properties, but are known to have theoretical limitations. Additionally, the translation of NL requirements into formal constraints typically requires significant manual effort. Recently, large language models (LLMs) have emerged as an alternative approach for formal reasoning tasks, but their effectiveness in verifying requirements over strings is less studied. In this paper, we introduce a hybrid approach that verifies the satisfiability of NL requirements over strings by using LLMs (1) to derive a satisfiability outcome (and a consistent string, if possible), and (2) to generate declarative (i.e., SMT) and imperative (i.e., Python) checkers, used to validate the correctness of (1). In our experiments, we assess the performance of four LLMs. Results show that LLMs effectively translate natural language into checkers, even achieving perfect testing accuracy for Python-based checkers. These checkers substantially help LLMs in generating a consistent string and accurately identifying unsatisfiable requirements, leading to more than doubled generation success rate and F1-score in certain cases compared to baselines without generated checkers.

Index Terms—large language model, satisfiability checking for requirements, test data generation, constraint solving

I. INTRODUCTION

Motivation. Large-scale software systems commonly involve numerous parties, from domain experts to technical team members and end-users. For the successful development of such systems, it is necessary to ensure a smooth interaction between these parties, which becomes particularly challenging when considering their varying levels of domain expertise and technical knowledge. For this purpose, requirements are most commonly captured using natural language (NL).

A particularity of software systems is their reliance on strings, not only for the implementation of software artifacts

(e.g., through textual programs) but also for data manipulation. For instance, many systems manipulate strings in the form of an email address and a password given by the user as input. Additionally, strings must comply with well-formedness requirements (e.g., an email address must contain exactly one “@” character), which are often numerous and interconnected. Verifying the satisfiability of an individual requirement over strings may be feasible via manual analysis. However, with an increasing number of requirements, common for large-scale projects, verifying their satisfiability becomes a challenging task, particularly considering that requirements may be contradictory or ambiguous. Such challenges have been studied by existing work in the field of requirements engineering [1]–[5]. In fact, inconsistency and ambiguity detection in (formal) requirements, even beyond strings, is an active subfield of requirements engineering.

Problem statement. Existing approaches for satisfiability checking of requirements over strings are often formal: they rely on general-purpose SMT solvers, such as Z3 [6], or on string-specific constraint solvers, such as Ostrich [7]. Such approaches are sound and efficiently handle complex constraints over strings by either (1) detecting problems, i.e., unsatisfiability, in input requirements or (2) generating conforming strings as proof of satisfiability, which is particularly useful in the context of requirements-based testing. However, a *major assumption* of such approaches is to take a formal definition of requirements as input (e.g., using the SMT-LIB2 syntax), which is often unavailable (and hard to create by formal methods non-experts).

Given that string requirements are mainly written in NL, NL-based approaches are better suited for their interpretation. In recent years, large language models (LLMs) have emerged as a promising alternative for NL interpretation in general, with certain logic reasoning capabilities [8], particularly due to their strong few-shot generalization capabilities in data-scarce settings. However, existing research [9] has shown that the effectiveness of using LLMs to verify the satisfiability of (NL) string requirements is below that of formal approaches. Still, their inherent support for NL requirements, as well as their capacity of generating *realistic* strings as proof of consistency,

^{*}Both authors contributed equally to the research.

[†]Also affiliates with McGill University.

Partially supported by the FRQNT-B2X project (file number: 319955), IT30340 Mitacs Accelerate, and the Wallenberg AI, Autonomous Systems and Software Program (WASP), Sweden

remain *significant assets* vis-à-vis formal approaches.

A further drawback of LLM-based verification of requirements satisfiability is the lack of soundness guarantees. To remedy this, existing LLM-based approaches [10]–[12] rely on a trial-and-error method guarded by formal checks: LLM-derived outcomes are formally checked for correctness at each iteration and may only be returned if the check succeeds. On the one hand, checking may be performed using (declarative) SMT solvers, which have known theoretical limitations, e.g., supporting nested quantifiers, which do occur in problems over strings. For instance, checking if a string is a palindrome is complex since it requires the solver to check that *for each* character at position i of data d , there *exists* an identical character at position $d.length - i$. On the other hand, imperative methods such as Python programs may efficiently check that strings conform to complex requirements, with less theoretical limitations. However, they cannot be used to provide claims that the requirements are unsatisfiable.

Contributions: We introduce a hybrid generative approach to verify the satisfiability of NL requirements over strings. Our approach uses LLMs (1) to derive a satisfiability outcome, and (2) to generate checkers used to ensure the correctness of the satisfiability outcome derived in (1). Our approach generates two types of checkers: declarative checkers (i.e., SMT specifications to verify unsatisfiability of requirement sets) and imperative checkers (i.e., Python programs, which can more generally check data when a requirement is satisfiable). The specific contributions of the paper are as follows:

- We propose a novel approach that takes as input a collection of NL requirements over strings and either (1) detects unsatisfiable requirements or (2) generates a conforming string (instance) as a proof of satisfiability. (Section III)
- We integrate a divide-and-conquer approach for efficient handling of complex collections of string requirements. As a novel intermediate step, our approach derives checkers for verifying the satisfiability of individual NL requirements and leverages a formal solver within a feedback loop to improve accuracy. (Section IV)
- Our approach relies on checkers to examine the correctness of LLM-derived satisfiability outcomes. In the case where no correct outcome is derived, our approach degrades gracefully by returning the closest-to-sound outcome derived within the allocated budget. (Section V)
- We conduct experiments with four popular LLMs, (i.e., GPT-4o [13], GPT-4o-mini [13], Llama3.1-8b [14], and DeepSeek-V3 [15]). We investigate 340 NL requirement sets conforming to 12 categories of textual variables drawn from common programming exercises in technical interviews. (Section VI)

Added value: In contrast to approaches focused solely on data generation, our approach provides accurate outcomes for both satisfiable and unsatisfiable (string) requirement sets. As a key improvement, this paper generates verification outcomes by taking as input *only* the NL requirements, i.e., without the

need to manually formalize NL requirements. Additionally, our comparative evaluation gives insights on the strengths and weaknesses of various LLMs and checker types in providing correct verification outcomes for NL requirements.

II. BACKGROUND

A. Satisfiability checking of requirements over strings

This paper focuses on verifying the satisfiability of requirements (given as input) over strings by leveraging a consistent string data and checker generation approach. To illustrate this, let us consider, as an illustrating example, an extended version of the email address manipulation system introduced in Section I which has the following three requirements:

- R1 The system shall require an email address to contain exactly one “@” character.
- R2 The system shall require an email address not to end with a “.” character.
- R3 The system shall require an email to contain at least one “.” character after the “@” character.

We illustrate the versatility of requirements satisfiability verification by considering the above system in a requirements-based testing context. Given requirements $R1$, $R2$, and $R3$, a test engineer first selects a set of *test cases*, represented as conditions over the set of requirements. They then associate an *expected system behavior* to each test case (e.g., the system stores the email address if and only if the given input satisfies all system requirements). An example containing five such test cases is shown in Table I.

TABLE I: Test cases, their satisfiability, and corresponding test data

Test case	Expected Behavior	Satisfiability	Test data
$R1 \wedge R2 \wedge R3$	Store email	Yes	“ab@cd.com”
$\neg R1 \wedge R2 \wedge R3$	Show error	Yes	“ab@@cd.com”
$R1 \wedge \neg R2 \wedge R3$	Show error	Yes	“ab@cd.”
$R1 \wedge R2 \wedge \neg R3$	Show error	Yes	“ab@cdcom”
$R1 \wedge \neg R2 \wedge \neg R3$	Show error	No	-

Upon test case selection, as a first step, the engineer must check whether the test case is satisfiable, which is a challenging task to perform manually, even for domain experts. For instance, it is difficult to determine manually that a $R1 \wedge \neg R2 \wedge \neg R3$ test case is unsatisfiable, considering that if an email containing exactly one “@” character ($R1$) does not contain any “.” characters after the “@” ($\neg R3$), then it cannot end with a “.” ($\neg R2$). While SMT solvers may efficiently detect such an unsatisfiable case if the requirements are formalized, a key advantage of our approach is to derive *SAT/UNSAT* outcomes directly from NL requirements.

Once the satisfiability is determined, for each satisfiable test, the engineer must derive *textual test data* (i.e., a string) that would trigger the expected behavior in the underlying system. Deriving such data manually and ensuring its compliance with the test case specification is a daunting task, particularly with an increasing number of requirements. Our approach generates conformant test data automatically for each satisfiable test

case as a side-effect of the satisfiability verification, where the generated test data is the proof of satisfiability.

B. Constraint satisfaction problems over strings

Constraints over strings are common in the context of commercial software applications, e.g., through password and email restrictions. They are also addressed significantly in scientific research, through automated, string-specific formal reasoning techniques [7], [16]–[19] designed to solve collections of complex constraints. These approaches are evaluated over large benchmarks [20] and in the context of solver competitions [21], where string constraints are represented formally (e.g., in the SMT-LIB2 [22] format). In this context, the solvers are evaluated according to their achieved runtime and success rate.

While the constraint satisfaction problems contained in these benchmarks are valuable from a logical perspective, their relevance for real-world application domains such as requirements engineering has been less studied. Additionally, solvers take as input a formal representation of the problem, which is often unavailable upfront for most software systems.

C. Large language models (LLMs)

LLMs are powerful transformer neural networks [23] trained on vast amounts of data for handling various tasks involving natural language. Recent research highlights LLMs’ emerging reasoning capabilities, achieving notable performance on many logic reasoning benchmarks [8]. This positions them as promising alternatives to traditional constraint solvers for addressing constraint-solving problems involving text values.

Recent research has also focused improving LLM performance by optimizing the input NL prompt, known as *prompt engineering* [24]. Techniques such as leveraging emotional stimuli [25] and zero-shot chain-of-thought reasoning [26] have been shown to be effective across various LLMs.

While LLMs typically produce natural language output, recent advancements such as Open AI’s *structured outputs* [27] and LangChain’s *output parsers* [28] have enabled them to produce structured output. In this paper, we rely on LangChain to instruct the LLM to present final answers in various structured formats depending on the components.

III. APPROACH OVERVIEW

An overview of our proposed satisfiability verification approach for NL requirement is shown in Figure 1, where the blue and orange boxes represent input, intermediate, and output artifacts, while the red boxes represent processes. Our approach takes as input a conjunction of *NL requirements* such as the ones introduced in Section II-A. Our approach outputs a *Verification Outcome* $v_i \in \{UNSAT, SAT, UNKNOWN\}$ assessing the satisfiability of the input requirements conjunction.

As a first step, our approach generates a *Verification Outcome Checker* (checker) for each input NL requirement. A checker is a process that takes as input (1) the NL requirements and (2) an intermediate *UNSAT* or *SAT* verification outcome for the requirements derived by an intermediate,

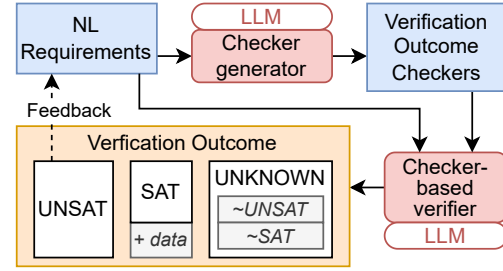


Fig. 1: Approach overview

not-necessarily-sound approach (e.g., LLMs), as detailed in Section V. It then checks (validates) the correctness of the intermediate verification outcome wrt. the requirements. The *checker generator* component is described in Section IV.

NL requirements and the generated checkers are then given as input to the *checker-based verifier*, detailed in Section V, which outputs a verification outcome for the conjunction of input NL requirements. When the requirements are determined to be satisfiable, the verifier returns *SAT*, as well as *string data* as a proof of satisfiability. When unsatisfiable NL requirements are detected, the verifier returns *UNSAT* and may pinpoint the problematic subset of the NL requirements through a *feedback* loop. Similar to existing heuristic-based search approaches [29], a budget is imposed on the feedback loop to limit the number of retries. If the verifier runs out of budget, it returns an *UNKNOWN* outcome. As a key particularity, our approach degrades gracefully when no sound *SAT* or *UNSAT* outcome may be provided: in this case, our approach returns the closest-to-sound outcome derived within the allocated budget.

IV. CHECKER GENERATOR

Figure 2 shows a detailed workflow of our proposed divide-and-conquer *Checker generator* that takes as input a set of NL requirements and derives a checker for each requirement within that set.

Types of checkers. In this paper, we consider two types of checkers, namely declarative checkers based on SMT solvers and imperative checkers based on Python scripts. On one hand, a *declarative checker* is the formalization of the input NL requirement set into an SMT-solver-friendly (static) specification. This specification is fed to an SMT solver, which can verify its satisfiability. Declarative checkers may also be used to check the consistency of a generated string by integrating the string value as a constraint within the checker specification. On the other hand, an *imperative checker* consists of, for each requirement, a function where the requirement is formalised using standard software language constructs. The function takes as input a string and returns a boolean value indicating whether the string satisfies the requirement formalization.

Requirements splitting. As an initial step, the checker generator *splits* the NL requirements into a set of *requirement batches* which will be converted into checkers as a contiguous unit. Splitting may be done arbitrarily, and will result in a batching that lies between the following extremes (which we

assess in our evaluation): (1) *complete split*, which yields batches containing a single requirement, and (2) *no split*, which considers the entire requirement set as a single batch.

Such an input batching approach is widely used to enhance efficiency across various domains, including neural network training and testing [30], as it often helps reduce neural network’s memory usage and/or training time. For our checker generation, batching reduces the number of LLM calls required to generate all checkers and may provide context when requirements are inter-related. However, for this paper, we hypothesize that complete splitting reduces the complexity of checker generation despite an increased number of LLM calls. We will demonstrate this in Section VI.

Requirements-to-checker conversion. Each batch of NL requirements is then converted into a corresponding batch of checkers using an LLM.

For generating imperative Python checkers, we employ *zero-shot* prompting, where the prompt consists of a Python function signature and a docstring comment containing the NL requirement. This approach specifies the intended function behavior, similar to how the HumanEval dataset prompts LLMs in their code generation benchmark [31].

For generating declarative SMT checkers, we provide one example in the prompt consisting of a batch of NL requirements along with an example pair of NL requirements and their corresponding SMT checker expression in string format, and it focuses on deriving exactly one SMT expression string as checker for each requirement within the batch. This example guides the LLM on the expected output format, increasing the likelihood that the generated result adheres to the desired format.

To enhance the LLM’s performance, we also incorporate several best practices into the prompt design, namely *role description*, *chain-of-thought prompting* [32], and *emotional stimuli* [25].

Checker soundness evaluation. Once a batch of checkers is derived, it is subjected to a syntactic *soundness evaluation*, which may be augmented with semantic soundness. Note that for a given batch of checkers, the soundness of each individual checker is evaluated separately. If all checkers within the batch are deemed to be sound, the batch is ready to be merged with other batches and returned as a contiguous set. Otherwise, if there exist unsound checkers in a batch, feedback information is gathered from the soundness evaluation (e.g., which specific checkers are unsound), then fed back into the LLM-based requirement-to-checker converter. Once all batches of checkers are evaluated to be sound, they are merged into a complete set of verification outcome checkers to be returned.

Since checkers are formal textual artifacts, their soundness is two-fold: syntactic and semantic. On one hand, checkers must conform to the syntactic rules of the corresponding formal language, e.g., Python syntax for imperative checkers. This may be evaluated efficiently by language-specific parsers.

On the other hand, ensuring *semantic* soundness of checkers (i.e., whether the checker indeed checks the correct property) is complex and formalism-dependent. For declarative checkers,

this may be assessed via formal equivalence checks comparing them to ground truth implementations, if available. When an individual NL requirement is satisfiable, semantic correctness, for both declarative and imperative checkers, may be approximated through testing, as proposed by existing research [31]. In this setting, valid- and invalid-labeled data is fed into the checker and the derived checker outcome is compared to the ground-truth labeling.

V. CHECKER-BASED NL REQUIREMENTS VERIFIER

Figure 4 shows a detailed workflow of our proposed checker-based NL requirements verifier component, which takes as input the original *NL requirements* and the *verification outcome checkers* derived in accordance to Figure 2 and returns a verification outcome for the input requirements. As an initial step, we leverage an LLM to derive an *LLM-derived verification outcome* (LVO) for the input NL requirements. For this purpose, we adopt *zero-shot* prompting by describing the requirement verification in the prompt. We then ask the LLM to output string data that satisfies all the NL requirements if it is possible to do so. Otherwise, we ask the LLM to output *UNSAT*. We follow the similar prompting best practices as described earlier in Section IV. Figure 3 shows an excerpt of the prompt template used for generating LVOs with feedback.

Correctness evaluation. Once the LVO is derived, we *evaluate its correctness* by leveraging the checkers given as input. When the LLM claims that the input requirements are unsatisfiable, our approach relies on declarative checkers (e.g., based on SMT solving) to assess its correctness. When the LLM provides a proof-of-consistency string as output, both declarative and imperative (e.g., a Python script) checkers may be leveraged for correctness evaluation. This separation is due to the theoretical limits of imperative checkers: they rely on queries and operations over data to provide a correctness outcome, which is not possible if no data is provided by the LLM (as in the *UNSAT* case).

If the checker confirms the correctness of the LVO, the latter (i.e., either *UNSAT* or *SAT+data*) is returned by the process. Otherwise, the checker may return an *incorrect* or an *unknown* outcome. An *incorrect* outcome triggers a feedback loop, where the *quality* of the current LVO is assessed *relative to the configured level of feedback information* (e.g., the specific requirements that were violated), then the LVO and its feedback is *saved* for future use. The quality of an LVO is determined as a side-effect of its correctness evaluation. In the case where the LVO is *SAT+data* and the checker returns *UNSAT*, we associate the quality of the LVO to the ratio of input requirements that the *data* satisfies. If the LVO is *UNSAT* and the checker returns *SAT*, then the quality of the *UNSAT* LVO is set to a fixed value. Note that the SMT checker may also produce data for the input requirements. We do not wish to return the checker-generated data directly since data generated by SMT checkers often lacks realism, and the LLM-generated checkers themselves may also be incorrect.

Declarative checkers may also return an *unknown* correctness outcome due to limitations of the checker (e.g., the SMT

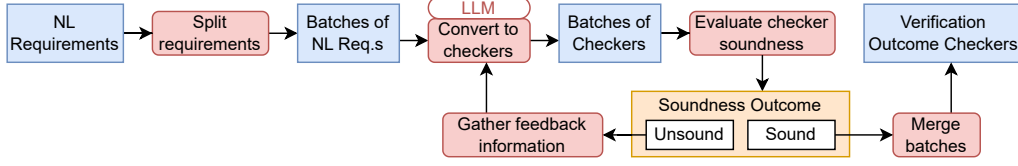


Fig. 2: Overview of our divide-and-conquer approach for generating verification outcome checkers

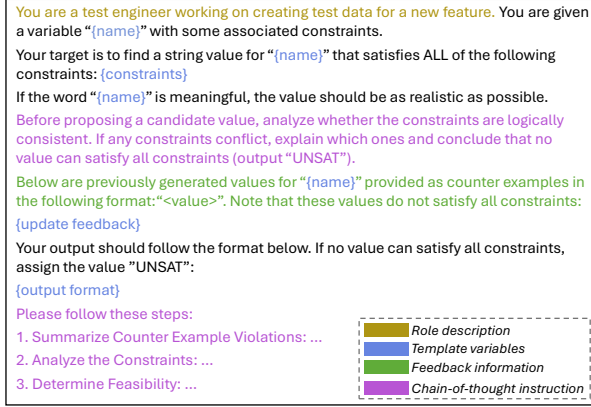


Fig. 3: LVO generation with feedback prompt template excerpt

process). Thus, the feedback loop is not initiated. Instead, the best saved LVO is determined, as detailed below, and returned.

Budget check. Once the current LVO is saved, we check the remaining budget (e.g., runtime, number of LLM calls, or number of tokens). If a budget remains, we continue the feedback loop by optionally incorporating saved LVOs and/or their feedback into the update_feedback field of the subsequent LVO generation prompt. Section VI discusses the different levels of feedback information we used in our experiment.

If, at the current stage, no budget remains, we determine the best LVO among all saved LVOs (i.e., the closest-to-sound) and return an UNKNOWN verification outcome. In practice, users may proceed with such fallback output even when full correctness was not established. This behavior is particularly useful in scenarios such as interactive workflows, where quick iterations may be prioritized over absolute certainty.

We determine the best saved LVO as follows: (1) If there are more saved UNSAT LVOs than there are saved SAT+data LVOs, then the best saved LVO (the most likely NL requirements verification outcome) is UNSAT (~UNSAT). (2) Otherwise, the best saved LVO is the SAT+data outcome with the highest ratio of input NL requirements satisfied. In this case, the NL requirements' LVO is most likely SAT (~SAT).

Failure mode. A critical failure mode occurs when both the LLM output and its checker are flawed but agree, leading to incorrect results. While feedback helps, it cannot fully eliminate the risk. Future work should explore safeguards like redundant or cross-validated checkers.

VI. EVALUATION

We systematically evaluate our proposed LLM-based approach for requirement satisfiability verification. The proposed framework consists of two main components: the checker generator and the checker-based verifier. To assess the performance of each component individually, RQ1 evaluates the checker generator using different LLMs, while RQ2 measures the LLM verifier's theoretical performance in an experimental setting where ground-truth checkers are available. However, these two research questions focus on isolated components rather than the overall system. Therefore, RQ3 then evaluates how well the approach performs in an end-to-end, realistic deployment setting (i.e., without ground truth checkers), where LLMs are used to (1) generate the checkers, and to (2) use them to produce LVOs based solely on NL input. Specifically, we address the following research questions:

- RQ1 What is the soundness of declarative and imperative checkers generated by LLMs?
- RQ2 How do the feedback mechanism and checker type impact the quality of LVOs from the LLMs?
- RQ3 What is the end-to-end performance of the framework when LLMs generate both checkers and LVOs?

A. Evaluation setup

a) *LLMs*: We compare four popular LLMs that differ in size and accessibility. **GPT-4o-mini** and **GPT-4o** [13] are powerful LLMs recently developed by OpenAI. Compared to their predecessors, they demonstrate superior performance as well as improved speed and cost-efficiency across a range of language understanding and reasoning benchmarks. Additionally, these models exhibit strong code-generation capabilities and have been integrated into various software development tools, including GitHub Copilot [33].

Despite their impressive capabilities, GPTs are closed-source and accessible only through APIs, which may raise concerns regarding data security and privacy. To address these potential issues, we include two open-source LLMs in our evaluation. **DeepSeek V3** [15], a mixture-of-experts LLM, achieves performance comparable to GPT-4o while substantially reducing training costs. **Llama3.1 8b** [14] is a smaller-scale model offering a balanced trade-off between performance and computational efficiency. It has been shown to achieve performance comparable to larger-scale LLMs in various tasks [14] while remaining deployable on consumer-grade GPUs.

b) *Checkers*: In this experiment, we explore two types of checkers for string data validation and propose a hybrid approach that leverages the strengths of both.

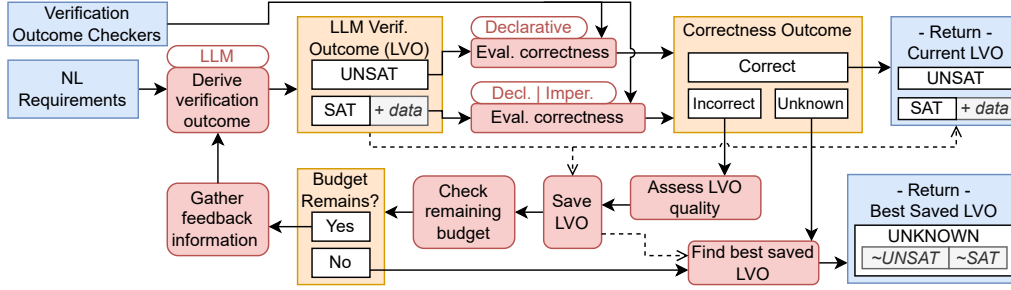


Fig. 4: Overview of our checker-based NL requirements verification approach

(*Declarative*) SMT solvers are often used for handling constraint satisfaction problems involving strings. They can verify whether a given string complies with a set of *formal* constraints and detect the unsatisfiability of these constraints. In this study, we adapt the **CVC5** solver to validate strings against formal constraints. Nevertheless, our approach is independent of the solver used, allowing other SMT solvers for strings, such as **Z3** and **Z3str**, to be used as well. However, the declarative nature of SMT solvers makes them challenging for handling complex constraints involving strings. Our preliminary experiments indicate that these solvers frequently time out on recursive constraints (e.g., placement of parentheses in mathematical formulas) while exhibiting inherent limitations with universal quantifiers [34].

(*Imperative*) These limitations can effectively be mitigated by imperatively checking strings against constraints. In our experiments, we select **Python** as the language for implementing imperative checkers, primarily due to the exceptional code-generation capabilities of most LLMs using Python. Nevertheless, imperative checkers have an intrinsic limitation: they can only confirm that a specific string conforms to requirements while they are unable to determine if the requirements themselves are unsatisfiable.

For our experiment, we ask an LLM to generate declarative checkers as an SMT string expression, which we use with Python’s **Z3** API to verify against our LVOs. The imperative checkers are generated as Python function implementations, which can be parsed natively to Python executables. An example of checker generation is shown in Figure 5.

Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

Declarative checker.

```
(and (str.contains s "@" ) (not (str.in.re s (re.++ (re.* re.allchar) (str.to.re "@" ) (re.* re.allchar) (str.to.re "@" ) (re.* re.allchar))))))
```

Imperative checker.

```
def this_constraint(s: str) -> bool:
    return s.count("@") == 1
```

Fig. 5: An example requirement and its according checkers.

c) *Dataset*: To evaluate the LVO quality of our requirements verification approach, we use requirements over twelve categories of string variables drawn from common programming exercises in technical interviews, namely *Email*, *Name*, *Password*, *Url*, *Date*, *IBAN number*, *ISBN number*, *Arithmetic expression*, *Palindrome*, *Parentheses sequence*, *DNA sequence*, and *File path*. While these are common programming exercises, prior studies have shown that they remain challenging for LLMs [9]. Notably, the requirements in our case studies correspond to production rules derived from regular and context-free grammars, which require computational models up to pushdown automata for resolution.

From these requirements, we derive a total of 340 NL requirement sets (which include both satisfiable and unsatisfiable cases) by considering all possible combinations of the requirements and their negations using *equivalence partitioning* [35]. To ensure conditions are precise, we manually formalize these requirements.

As an example, let us consider the following requirement from the *ISBN number* category “The system shall require an ISBN number to contain exactly 10 characters, excluding hyphens”, which is itself satisfiable. When fed into the **GPT-4o** LLM for verification, the “1-234-56789-X” string is returned as a proof of satisfiability. Satisfiability is also confirmed by the **CVC5** SMT solver, which yields “0000000000” (a much less realistic value).

To assess the quality of verification outputs, we first derive a ground truth satisfiability label for each requirement set by manually translating the requirements to formal logical constraints (using the SMT-LIB2 syntax) and feeding them to the **CVC5** solver (with 5-second timeout). Among the 340 requirement sets within the dataset, 230 returned *SAT*, 54 returned *UNSAT*, and 56 returned *UNKNOWN* (which, upon manual inspection, yielded 53 *SAT* cases and 3 *UNSAT* cases). Therefore, in our final labeling, we have 283 *SAT* cases and 57 *UNSAT* cases.

d) *Implementation details*: To minimize variations caused by LLM generation, we set the temperature to 0 for open-source models and to 0.01 for GPT models. When controlled variation is needed, such as retrying without feedback, we use a low temperature of 0.1. We set the maximum retry budget to 2 LLM calls for the check generator and to 5 for the

checker-based verifier. For evaluation purposes, we map the most likely *UNSAT* ($\sim$ *UNSAT*) and *SAT* ($\sim$ *SAT*) outcomes directly to *UNSAT* and *SAT*, respectively. The anonymized artifact accompanying this paper is available online [36].

B. RQ1: Checker generation

a) *Setup*: Here, we investigate the soundness of LLMs to generate declarative (*SMT*) and imperative (*Python*) checkers from NL requirements under different settings. For both types of checkers, we provide feedback on their *syntactic soundness* by the corresponding parser during checker generation.

Compared approaches. We compare two batching methods for each requirement set, in accordance with Section IV: (1) the *BATCH* method yields two batches: one for all positive requirements, one for all negated requirements; (2) The *IND* method creates one independent batch for each requirement.

Metrics. We evaluate the quality of n generated checkers from both syntactic and semantic perspectives.

For *syntactic soundness*, we rely on Python and SMT parsers to verify the syntactic correctness of generated checkers. Formally, let $\mathcal{V}$ be the set of generated checkers and $\mathbb{P} : \mathcal{V} \rightarrow \{0, 1\}$ represent a parser function that returns 1 if the checker is syntactically valid and 0 otherwise. We define **syntax accuracy** as the ratio of syntactically valid checkers to the total number of checkers generated: $\frac{\sum_{v \in \mathcal{V}} \mathbb{P}(v)}{n}$. Such a metric is adequate to evaluate the usability of generated checkers (compared to, e.g., a finer-grain syntax accuracy metric that counts the syntax errors) since checkers must be syntactically error-free to be applicable in our use case.

We also define the *proportion of SMT checkers whose equivalence with ground truth checkers has been formally proven* as **formal accuracy**. Formally, for a set of checkers $\mathcal{V}$, the formal accuracy is defined as $\frac{|\{v | v \in \mathcal{V} \wedge v \equiv v_{gt}\}|}{n}$. This equivalence check is converted into an SMT formulation and subsequently verified using the CVC5 solver. Checkers are considered formally correct only if equivalence is *explicitly* proven; all solver results that yield *timeout* or *unknown* outcomes are disregarded. Formal accuracy is the most precise metric for evaluating correctness; however, it can only be computed using declarative checkers, as it requires the CVC5 solver to formally verify the equivalence of two SMT expressions. To the best of our knowledge, no existing method allows for the formal equivalence checking of Python functions.

To evaluate semantic soundness across both declarative and imperative checkers, we adopt a testing-based approach. Specifically, for each requirement, we manually construct five string samples that satisfy the requirement and five that violate it, resulting in a total of 10 testing samples per requirement. These samples are then validated using the generated checkers and compared against the ground truth labels. A checker is considered semantically correct if it produces identical outputs to the ground truth checker for *all* associated samples. We refer to the proportion of checkers meeting this criterion as **testing accuracy**. Formally, let S_v be the set of sample string data associated with the original requirement of

TABLE II: Accuracy of **Python** checker generation (in %)

LLM	Approach	Syntax Acc	Testing Acc
Llama3.1-8b	BATCH	100	100
	IND	100	98
GPT-4o-mini GPT-4o DeepSeek-V3	BATCH	100	100
	IND	100	100

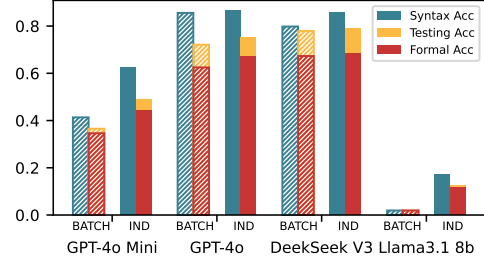


Fig. 6: Accuracy of generated **SMT** constraints (in %)

checker v , let v_{gt} denote the ground truth checker corresponding to v , and let $E(v, s)$ represent the execution output of checker v on sample s . Testing accuracy is defined as: $\frac{|\{v | v \in \mathcal{V} \wedge \forall s \in S_v : E(v, s) = E(v_{gt}, s)\}|}{n}$.

Testing accuracy provides an *over-approximation* of checkers' semantic correctness, as it relies on a non-exhaustive set of test samples. For **SMT** checkers, however, it is possible to *formally prove* semantic correctness by verifying their equivalence with the ground truth checkers.

b) *Results*: Table II and Figure 6 present the soundness of **Python** and **SMT** checkers generated by LLMs, respectively. For Python, all tested LLMs exhibit exceptional capability, achieving perfect syntax accuracy across both *BATCH* and *IND*. Moreover, three of the evaluated models (**DeepSeek-V3**, **GPT-4o**, and **GPT-4o-mini**) maintain perfect testing accuracy (100%), with only **Llama3.1-8b** slightly lower at approximately 98% in the *IND* setting. These results strongly suggest that Python is a highly suitable language for checker generation using LLMs, leveraging their strengths in Python code generation tasks. Indeed, prior research has already demonstrated LLMs' capability on challenging code generation benchmarks such as HumanEval [15]; generating Python-based checkers for string validation is then a comparatively less challenging task. However, it is important to note that Python checkers are not capable of validating unsatisfiability.

LLMs' capability is particularly important when generating **SMT** checkers. As shown in Figure 6, smaller-scale LLMs (**GPT-4o-mini** and **Llama3.1-8b**) achieve notably lower quality compared to larger models (**GPT-4o** and **DeepSeek-V3**). Specifically, outputs from **Llama3.1-8b** exhibit significantly lower quality, achieving less than 20% syntax and testing accuracy in *IND*. This clearly suggests that **Llama3.1-8b**'s smaller scale limits its ability to effectively generate SMT checkers. Conversely, **DeepSeek-V3** and **GPT-4o** achieve substantially higher accuracy, both with over 85% syntax accuracy and over 75% testing accuracy in *IND*. Interestingly, formal

accuracy is closely aligned with testing accuracy, suggesting that most of the checkers passing the sample-based tests can be formally proven correct. When comparing syntax correctness to semantic correctness, the ratio between syntax accuracy and semantic accuracy exceeds 65% in all evaluated cases and surpasses 70% in six out of eight cases. This observation suggests that while syntax accuracy is already relatively high, further enhancing syntax correctness could still positively influence semantic soundness.

Comparing BATCH and IND generation approaches shows that treating each requirement independently generally benefits the LLMs, with IND consistently outperforming BATCH across all tested LLMs. The accuracy gain from independent handling is particularly pronounced for smaller LLMs. For instance, **GPT-4o-mini** shows an over 10% increase in accuracy when using IND, compared to only around 3% improvement for **GPT-4o**. This indicates that larger LLMs can effectively handle multiple requirements simultaneously, making the simplified IND scenario less beneficial to them.

Answer to RQ1. Three LLMs achieve perfect accuracy (100% testing accuracy) in generating Python checkers, with **Llama3.1-8b** achieving near-perfect accuracy ($\geq 98\%$) across both BATCH and IND settings. For SMT checkers, LLM size significantly influences the soundness, as larger LLMs (over 75% in testing accuracy for **DeekSeek-V3**) consistently outperform smaller ones (less than 20% in testing accuracy for **Llama3.1-8b**). Moreover, treating each requirement independently (IND) consistently helps improve both syntactic and semantic soundness compared with batched input (BATCH).

C. RQ2: Checker-based verification

a) *Setup:* Existing studies suggest that applying an explicit validation stage for LLM generation may significantly improve the LLM's performance in various tasks [9]–[12]. Building upon this insight, this research question further explores how different feedback mechanisms and checkers influence the quality of LVO produced by LLMs in the context of requirement verification. Specifically, this RQ focuses on the theoretical performance of checker-based verification when a ground truth checker is present. In RQ3, the best theoretical settings are then evaluated in the end-to-end setting to understand the overall effectiveness in a realistic use case.

Compared approaches. We compare the direct LLM generation approach with various checkers and feedback methods. The **direct** approach validates the satisfiability of constraints with a single call to the LLM. In this approach, the NL requirements are fed into the LLM, and the derived LVO is returned directly (with neither a checker-based evaluation nor a feedback loop). In this case, a checker-based evaluation is irrelevant since the derived feedback cannot be leveraged.

For feedback approaches, we assess both imperative checkers in **Python** and declarative checkers using **SMT**. Moreover, we propose a **Hybrid** approach, which uses **Python** to validate generated strings and **SMT** to verify unsatisfiability decisions.

For each checker type, we examine three distinct feedback mechanisms: (1) **V** (validation), which follows a trial-and-error approach, where checkers provide no explicit feedback but ask the LLM to retry upon incorrect outcomes; (2) **VF** (validation with feedback), which explicitly incorporates incorrect LVOs from previous loop(s) as counterexamples within the generation prompt; and (3) **VFE** (validation with feedback and explanation), which further enriches feedback by specifying which of the natural language requirements did each counterexample fail, using validation results from each requirement's corresponding checker.

Metrics. We assess LVOs from different approaches based on the generation quality of the test data for SAT outcome and the quality of UNSAT decisions.

The Generation Success Rate (**GSR**) measures the percentage of requirement sets successfully verified by generating appropriate test data. Formally, let $\mathcal{D}$ be the dataset of requirements, $\mathcal{D}_{\text{sat}} \subseteq \mathcal{D}$ represent the subset of satisfiable requirements, v_{gt}^d denote the ground truth checker associated with each set of requirements $d \in \mathcal{D}$, and s_d represent the string generated by an approach for a requirement set d . Furthermore, let $\mathbb{E}(v, s) \in \mathbb{B}$ indicate the validation result of checker v on output s . The **GSR** is defined as:
$$GSR = \frac{|\{d | d \in \mathcal{D}_{\text{sat}}, \mathbb{E}(v_{gt}^d, s_d)\}|}{|\mathcal{D}_{\text{sat}}|}.$$

To evaluate the effectiveness of each approach in detecting unsatisfiable requirement sets, we compute standard precision (**P**), recall (**R**), and **F1**-score metrics using the unsatisfiable subset of the dataset, $\mathcal{D}_{\text{unsat}}$. Let $\mathcal{S}_{\text{unsat}}$ denote the set of outputs identified as unsatisfiable by an approach. These metrics are defined as: $P = \frac{|\mathcal{D}_{\text{unsat}} \cap \mathcal{S}_{\text{unsat}}|}{|\mathcal{S}_{\text{unsat}}|}$, $R = \frac{|\mathcal{D}_{\text{unsat}} \cap \mathcal{S}_{\text{unsat}}|}{|\mathcal{D}_{\text{unsat}}|}$, and $F1 = \frac{2 \times P \times R}{P + R}$.

b) *String generation results:* The measurement results for requirements verification across different checkers, feedback methods, and LLMs are presented in Table III. Underlined numbers indicate the best quality achieved by each checker by integrating with a specific LLM, while bold numbers highlight the top-performing results for each LLM. Results from the baseline (**direct**) approach reveal a significant difference in GSR depending on the scale of the LLM. Evidently, LLMs' capacity is directly correlated with output quality, as **GPT-4o** achieves the highest **direct** GSR at 78.45%, whereas smaller-scale LLMs, such as **Llama3.1-8b**, notably lag at 25.80%. Nevertheless, even the highest GSR from **direct** remains relatively low, highlighting the importance of additional validation techniques to enhance output quality.

Introducing checkers (**+V**) consistently enhances GSR across all evaluated LLMs compared to the **direct** approach. Notably, the GSR for **GPT-4o** increases from 78.45% to 91.87% when using the **Python** checker. Smaller models exhibit even greater relative improvement; for example, the GSR of **Llama3.1-8b** more than doubles from 25.80% to 55.48% with validation by SMT checkers. These results show the effectiveness of checkers in enhancing the capability of smaller LLMs to produce valid string outputs.

Interestingly, incorporating feedback with counterexamples

TABLE III: Quality of LVOs from compared approaches on NL requirements verification with ground truth checkers (in %)

checker	Methods	GPT-4o-mini				GPT-4o				DeekSeek-V3				Llama3.1-8b			
		GSR	P	R	F1	GSR	P	R	F1	GSR	P	R	F1	GSR	P	R	F1
Python	Direct	66.43	58.33	61.40	59.83	78.45	90.32	49.12	63.64	72.08	63.27	54.39	58.49	25.80	35.29	33.33	34.29
	+V	73.50	47.37	63.16	54.14	91.87	87.50	49.12	62.92	89.75	70.45	54.39	61.39	38.16	36.00	31.58	33.64
	+VF	71.38	50.77	57.89	54.10	90.81	96.67	52.63	68.16	82.33	66.00	57.89	61.68	38.87	34.15	26.32	29.72
	+VFE	74.56	52.31	59.65	55.74	91.52	90.62	50.88	65.17	84.81	62.00	54.39	57.94	45.94	46.67	36.84	41.18
SMT	+V	74.20	60.56	75.44	67.19	86.93	69.05	50.88	58.59	86.93	70.00	61.40	65.42	55.48	70.91	68.42	69.64
	+VF	75.97	65.82	91.23	76.47	87.28	82.26	89.47	85.71	84.10	80.60	94.74	87.10	48.76	71.43	98.25	82.72
	+VFE	74.56	69.33	91.23	78.79	89.75	81.54	92.98	86.89	89.75	79.41	94.74	86.40	46.64	68.42	91.23	78.20
	+V	77.39	63.08	71.93	67.21	92.23	93.55	50.88	65.91	91.87	85.37	61.40	71.43	53.36	77.50	56.14	65.11
Hybrid	+VF	78.09	72.00	94.74	81.82	90.11	96.15	87.72	91.74	90.81	87.10	94.74	90.76	51.94	80.95	91.23	85.78
	+VFE	77.74	74.29	92.98	82.59	92.23	96.36	92.98	94.64	91.87	89.83	96.49	93.04	57.24	80.60	94.74	87.10

(+VF) does not consistently enhance GSR and, in some instances, slightly decreases it. For example, using the **SMT** checker, the GSR notably drops from 55.48% to 48.76%. This reduction could be attributed to confusion caused by counterexamples lacking explanations, making it difficult for LLMs to identify why specific inputs fail validation. This hypothesis is reinforced by the improvements when explanatory feedback (+VFE) is introduced, resulting in enhanced GSR in 7 out of the 12 tested scenarios compared to both +V and +VF.

When comparing the effectiveness of different checkers, **Python** generally yields improvements similar to those of **SMT** for smaller LLMs (**GPT-4o-mini** and **Llama3.1-8b**). We hypothesize that smaller LLMs struggle with more complex requirements where Python checkers could be more beneficial. Indeed, **Python** consistently outperforms **SMT** for larger models, particularly **GPT-4o**, achieving the highest GSR of 91.87%, compared to 89.75% with **SMT**. Furthermore, combining **Python** and **SMT** into a **Hybrid** checker results in even better LVO quality, providing the highest GSR of 92.23% across all LLMs and feedback configurations evaluated.

c) *Requirement verification results:* When evaluating the effectiveness of the approaches in verifying the satisfiability of requirement sets, the **Python** checker alone does not consistently improve the generation F1-score, as it cannot validate the unsatisfiability (*UNSAT*) decisions made by LLMs. Conversely, the **SMT** checker can effectively verify these *UNSAT* decisions, yielding significant improvements over the **direct** approach. Notably, incorporating explanatory feedback (+VFE) enhances precision for three out of four LLMs and improves recall and F1-scores across all evaluated models. Moreover, the **Hybrid** checker uses Python's better capability to detect invalid string outputs, further improving output quality and achieving higher F1-scores compared to the **SMT** checker alone in the +VFE scenario. This improvement is more notable for smaller LLMs; for example, the F1-score of **Llama3.1-8b** increases substantially from 34.29% to 87.10% using the **Hybrid+VFE** approach.

When comparing feedback methods, providing more informative feedback consistently helps LLMs identify inconsistencies within sets of requirements. Introducing validation alone (+V) already enhances the F1-score in identifying unsatisfiable requirement sets, improving it in 8 out of 12 cases. However, because this method does not offer explicit feedback, quality gains remain limited. Adding counterexamples (+VF) further

enhances the output quality, yielding gains in 10 out of 12 scenarios compared to validation alone. Supplementing these counterexamples with explanations (+VFE) results in the highest overall LVO quality, further improving the F1-score in 8 out of 12 scenarios compared to +VF cases.

Answer to RQ2. Incorporating feedback from ground truth checkers substantially improves LVO quality from LLMs in terms of both GSR and verification F1-scores. Specifically, the **Hybrid** checker, combined with counterexamples and explanations (+VFE), achieves the best results, providing the highest GSR (57.24% to 92.23%) in three out of four LLMs and the best verification F1-scores (82.59% to 93.04%) across all evaluated LLMs.

D. RQ3: End-to-end evaluation

a) *Setup:* In this research question, we integrate the previously evaluated **checker generator** from RQ1 and **checker-based verifiers** from RQ2 to comprehensively assess the overall system.

Compared approaches. We evaluate the combined system of the best configurations identified in previous research questions for the **checker generator** and the **checker-based verifier**. Specifically, for the **checker generator**, we select the checkers generated by **IND** across all evaluated LLMs. For the **checker-based verifier**, we adopt the **Hybrid+VFE** configuration, which integrates the advantages of both declarative and imperative validation methods and provides the most detailed feedback. We assess the effectiveness of these combined configurations across all LLMs, comparing their output quality against the baseline results obtained without any checker or feedback loop (**Direct**, as presented in RQ2).

Metrics. We evaluate the LVO quality from the integrated system using the same metrics as defined in RQ2: **GSR**, **P**, **R**, and **F1**. Since RQ2 configurations use ground truth checkers, their corresponding metric values represent an upper bound on achievable LVO quality for the complete system. Therefore, in this RQ, we report the *ratio* of the measured metrics to these upper-bound values for each metric. Specifically, given a measured metric value m and its corresponding upper-bound value $\hat{m}$ (obtained using ground truth checkers), we calculate and plot the ratio $\frac{m}{\hat{m}}$ to illustrate how closely each evaluated configuration approaches the upper-bound metric values.

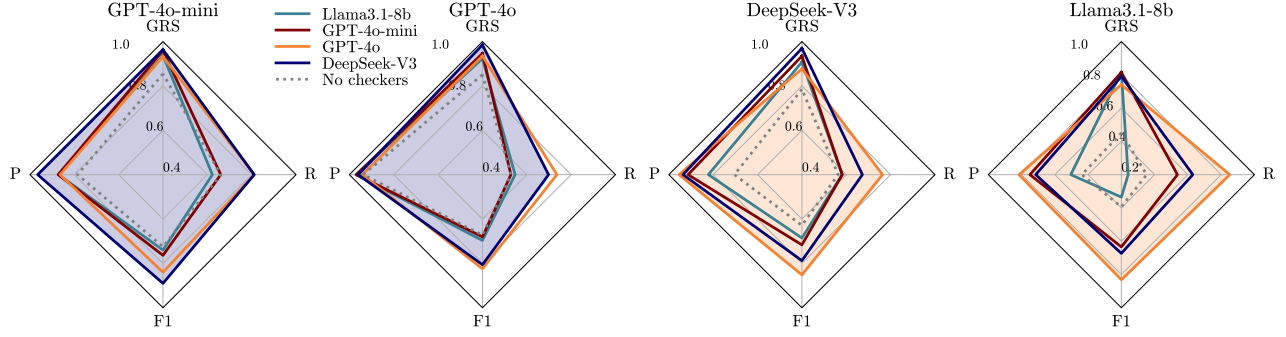


Fig. 7: Metric ratio of different LLM-based verifiers using LLM-generated checkers compared with the ground truth checkers; each figure represents a different LLM-based verifier; each line represents a checker generation method; the filled color shows the best checker generation method for each verifier

b) Results: Figure 7 presents radar plots illustrating the metric ratio of the system across various LLM combinations compared against the baseline approach without validation (**Direct** from RQ2). Specifically, the center of each radar plot corresponds to a metric ratio of 0.4 for **GPT-4o-mini**, **GPT-4o**, and **DeepSeek-V3**, whereas the center represents a ratio of 0.2 for **Llama3.1-8b**.

The quality of LVOs generated using the feedback approaches significantly outperforms the configuration without checkers (**Direct**) across almost all metrics and configurations. The only exceptions occur when checkers generated by **Llama3.1-8b** are used with either the **GPT-4o-mini** or **Llama3.1-8b** verifiers, likely due to the limited quality of these generated checkers combined with the lower capability of these smaller LLMs. Across all configurations, the **GSR** consistently exceeds the baseline without checkers and approaches the ground-truth checker, particularly when using checkers generated by **DeepSeek-V3** combined with verifiers such as **GPT-4o** and **DeepSeek-V3**, achieving ratios over 95%. However, verification **F1**-score remains considerably below their upper bounds, with the best case achieving around 88% when combining the **GPT-4o-mini** verifier with checkers generated by **DeepSeek-V3**. This performance gap likely results from the substantially lower quality of generated SMT checkers compared to Python checkers, highlighting the need for further improvements in SMT checker generation methods.

When comparing output quality across different checkers, higher checker quality consistently correlates with improved overall output quality. Specifically, verifiers using the highest-quality checkers generated by **GPT-4o** and **DeepSeek-V3** generally achieve the best results. Notably, **DeepSeek-V3**-generated checkers yield the highest **GSR** for three out of four verifiers, and **GPT-4o**-generated checkers produce the best **F1**-scores in three out of four verifiers. Interestingly, even though checkers generated by **Llama3.1-8b** have significantly lower SMT checker quality compared with other LLMs (see RQ1), they still enhance LVO quality across all metrics for the **GPT-4o** and **DeepSeek-V3** verifiers, and improve **GSR**, **P**, and **F1** metrics for the **GPT-4o-mini** verifier compared

to scenarios without checkers. This finding suggests that our feedback system generally shows robustness against variations in the quality of generated checkers.

Answer to RQ3. Combining an LLM-based checker generator with a checker-based verifier significantly improves output quality compared to direct generation without checkers, achieving over 95% GSR ratio in three verifiers and over 60% F1-score ratio in all verifiers. Enhancing the quality of individual components, such as using higher-quality checkers and more capable verifier LLMs, directly translates into improved overall performance in the final LVO. This finding highlights the modularization of the system for achieving better overall outcomes.

E. Threats to Validity

Internal validity: The authors manually convert each requirement set into checkers and testing samples, which could introduce biases or inaccuracies. To mitigate potential issues arising from manual effort, we cross-check the created checks and samples to make sure they are consistent with each other. Moreover, the dataset is imbalanced and contains more *SAT* cases than *UNSAT* cases. To mitigate bias caused by this imbalance, we evaluate different metrics separately for each category. Additionally, we control for variations in LLM outputs by setting temperature parameters to zero for open-source models (e.g., **Llama3.1-8b** and **DeepSeek-V3**) and to a minimal value (0.01) for GPT-based models, except in cases where controlled variation is necessary (such as the **+V** scenario), in which case a temperature value of 0.1 is used.

External validity: The collected requirements in the dataset may be impacted by selection bias. Therefore, our findings may differ when generalized to other contexts. Specifically, the original requirements may be present in the training data of LLMs, as they are derived from common programming exercises. However, the dataset includes all possible combinations of these requirements and their negations. Such unusual cases are less likely to occur in the training dataset. Future studies

are necessary to evaluate whether our approach generalizes well to more complex, industry-scale requirements.

Construct validity: We evaluate the quality of generated checkers based on semantic and syntactic correctness, consistent with general practice in code generation [37]. We assess the quality of LVOs on standard success rate for generation, and precision/recall/F1-score for classification.

VII. RELATED WORK

LLM for requirements engineering. LLMs are becoming increasingly popular in various stages of requirements engineering, including elicitation, modeling, and verification [38].

Ronanki et al. [39] explore the strengths and limitations of LLMs in the elicitation process. Görer and Aydemir [40] propose an LLM-assisted approach that yields interview questions for identifying new requirements. In requirement modeling, existing work has studied LLM’s ability in generating domain models [41], [42] and goal models [43], [44]. LLMs have also been effectively used for helping identify ambiguities or inconsistent sets of requirements [1], [2].

In this paper, we leverage LLMs for verification of NL requirements for satisfiability. As a key side effect, when requirement sets are satisfiable, our approach generates a conformant and realistic string instance as a proof of satisfiability, which is relevant as input data for requirements-based testing.

LLM for test data generation. Due to their impressive NL generation capability, LLMs have been increasingly applied to test data generation tasks (e.g., for testing graphical user interfaces [45], fuzz testing [46], validating SMT solvers [47]) from (satisfiable) requirements given as input. However, less focus has been placed on (1) satisfiability of requirements and on (2) assessing requirements over strings.

Existing research [48] takes as input user-written constraints for string variables and generates corresponding test cases. In contrast, our approach verifies the input requirements for satisfiability and outputs consistent test data when the requirements are satisfiable. Similarly, Babikian et al. [9] propose a textual test data generation approach from input NL requirements over strings. However, their approach relies on manual formalization of NL requirements to ensure output correctness, which is a significant drawback compared to our proposed approach.

Iterative generation with LLMs. Recent advances in LLMs have demonstrated impressive performance in various tasks. However, complex reasoning problems still pose significant challenges, motivating approaches that decompose complex tasks into iterative sub-tasks [49]. The Tree-of-Thought framework [49], [50] enables LLMs to approach decision-making incrementally, with the ability to trace back and revise decisions if necessary. Building upon this, the Graph-of-Thought framework [51] allows LLMs to organize their reasoning in a graph structure, enabling the parallel execution of multiple subtasks. More recently, multi-agent collaboration, employing multiple LLMs for distinct subtasks, has gained popularity [52]–[55]. Such methods leverage the strengths of

different LLMs and specialized prompting techniques tailored for each subtask.

Iterative LLM-based approaches have been effectively adapted to software engineering tasks such as code generation [52], [56], test generation [57], and software modeling [58]. In requirement engineering, specifically, iterative methods have integrated retrieval-augmented generation for enhanced requirement traceability and compliance checking [59].

In this paper, we propose an iterative approach using LLMs for requirement satisfiability verification. Our approach can also be seen as a multi-agent system, wherein one agent (checker generator) generates verification checkers to validate the verification outputs produced by another agent (checker-based verifier).

VIII. CONCLUSION

In this paper, we propose an approach that leverages LLMs, SMT solvers (i.e., CVC5), and imperative programming (i.e., Python) for the verification of NL requirements for satisfiability through consistent string and checker generation. Our approach takes as input a collection of NL requirements over strings and either (1) detects unsatisfiable requirements or (2) generates a conforming string instance as a proof of satisfiability. We experimentally show that checker generation is improved by assessing each requirement individually, while requirements verification benefits from integrating declarative and imperative checkers with detailed feedback. These findings allow us to achieve near-ground-truth performance without manual intervention, e.g., NL requirement formalization.

As future work, we plan to extend our satisfiability verification approach for string requirements to more complex textual formalisms, such as domain-specific textual languages. While this work focuses on string requirements, the underlying idea of combining checker generation with data generation is expected to generalize to a broader range of requirement types, provided that a corresponding formal specification language and constraint satisfiability solving approach are available. For instance, our approach may be applicable to the satisfiability verification of NL requirements on the safety of autonomous vehicles, where such requirements are represented as numeric conditions over the position and assigned path of each vehicle in a traffic scenario [60]. We plan to explore this extension in future work. Further improvements in SMT checker generation could be explored by incorporating semantic soundness feedback during the generation process. Additionally, we plan to improve the feedback loop proposed in Figure 4 by integrating stochastic mutations over a collection of simultaneous LVOs, in analogy to a genetic algorithm.

REFERENCES

- [1] N. Feng, L. Marsso, S. G. Yaman, I. Standen, Y. Baatartogtokh, R. Ayad, V. O. De Mello, B. Townsend, H. Bartels, A. Cavalcanti *et al.*, “Normative requirements operationalization with large language models,” in *RE 2024*. IEEE, 2024, pp. 129–141.
- [2] A. Fantechi, S. Gnesi, L. Passaro, and L. Semini, “Inconsistency detection in natural language requirements using ChatGPT: a preliminary evaluation,” in *RE 2023*. IEEE, 2023, pp. 335–340.

- [3] M. B. Hosseini, J. Heaps, R. Slavin, J. Niu, and T. Breaux, "Ambiguity and Generality in Natural Language Privacy Policies," in *2021 IEEE 29th International Requirements Engineering Conference (RE)*, 2021, pp. 70–81.
- [4] D. Giannakopoulou, T. Pressburger, A. Mavridou, and J. Schumann, "Generation of Formal Requirements from Structured Natural Language," in *Requirements Engineering: Foundation for Software Quality*, N. Madhavji, L. Pasquale, A. Ferrari, and S. Gnesi, Eds. Springer International Publishing, 2020, pp. 19–35.
- [5] D. M. Berry and E. Kamsties, "Ambiguity in Requirements Specification," in *Perspectives on Software Requirements*. Springer US, 2004, pp. 7–44.
- [6] L. De Moura and N. Björner, "Z3: An efficient SMT solver," in *TACAS 2008*. Springer, 2008, pp. 337–340.
- [7] T. Chen, M. Hague, A. W. Lin, P. Rümmer, and Z. Wu, "Decision procedures for path feasibility of string-manipulating programs with complex operations," *OSTRICH String Constraint Solver and Results*, vol. 3, no. POPL, pp. 49:1–49:30, Jan. 2019.
- [8] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," in *ACL 2023*. ACL, 2023, pp. 1049–1065.
- [9] A. A. Babikian, B. Chen, and G. Mussbacher, "Exploring large language models for requirements on string values," in *Proceedings of the 2nd IEEE/ACM Workshop on Multi-disciplinary, Open, and RElevant Requirements Engineering*, 2025. [Online]. Available: <https://mo2re.github.io/assets/preprints/BCM-LLMsForStringRequirements-MO2RE25.pdf>
- [10] Y. Li, J. Parsert, and E. Polgreen, "Guiding enumerative program synthesis with large language models," in *International Conference on Computer Aided Verification*. Springer, 2024, pp. 280–301.
- [11] E. First, M. N. Rabe, T. Ringer, and Y. Brun, "Baldur: Whole-proof generation and repair with large language models," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1229–1241.
- [12] J. H. Kirchner, Y. Chen, H. Edwards, J. Leike, N. McAleese, and Y. Burda, "Prover-verifier games improve legibility of llm outputs," *arXiv preprint arXiv:2407.13692*, 2024.
- [13] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [15] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [16] Y.-F. Chen, D. Chocholatý, V. Havlena, L. Holík, O. Lengál, and J. Síf, "Solving string constraints with lengths by stabilization," *Proceedings of ACM Programming Languages*, vol. 7, no. OOPSLA2, Oct. 2023.
- [17] K. Lotz, A. Goel, B. Dutertre, B. Kiesl-Reiter, S. Kong, R. Majumdar, and D. Nowotka, "Solving String Constraints Using SAT," in *CAV 2023*. Springer, 2023, pp. 187–208.
- [18] H. Wu, Y.-F. Chen, Z. Wu, B. Xia, and N. Zhan, "A decision procedure for string constraints with string/integer conversion and flat regular constraints," *Acta Informatica*, vol. 61, no. 1, pp. 23–52, Mar. 2024.
- [19] S. Krings, J. Schmidt, P. Skowronek, J. Dunkelau, and D. Ehmke, "Towards Constraint Logic Programming over Strings for Test Data Generation," in *Declarative Programming and Knowledge Management*. Springer, 2020, pp. 139–159.
- [20] "SMT-LIB The Satisfiability Modulo Theories Library," <https://smt-lib.org/benchmarks.shtml>.
- [21] "SMT-COMP 2024," <https://smt-comp.github.io/2024/>.
- [22] C. Barrett, P. Fontaine, and C. Tinelli, "The SMT-LIB standard: Version 2.6," Department of Computer Science, The University of Iowa, Tech. Rep., 2017.
- [23] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *NIPS*, 2017.
- [24] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," *arXiv preprint arXiv:2402.07927*, 2024.
- [25] C. Li, J. Wang, Y. Zhang, K. Zhu, W. Hou, J. Lian, F. Luo, Q. Yang, and X. Xie, "Large language models understand and can be enhanced by emotional stimuli," *arXiv preprint arXiv:2307.11760*, 2023.
- [26] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *NeurIPS 2022*, pp. 22 199–22 213, 2022.
- [27] OpenAI, accessed: 2025-03-10. [Online]. Available: <https://openai.com/index/introducing-structured-outputs-in-the-api>
- [28] H. Chase and contributors, "LangChain," 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [29] P. Dinges and G. A. Agha, "Solving complex path conditions through heuristic search on induced polytopes," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE-22)*, Hong Kong, China, November 16 - 22, 2014. ACM, 2014, pp. 425–436.
- [30] I. Kandel and M. Castelli, "The effect of batch size on the generalizability of the convolutional neural networks on a histopathology dataset," *ICT Express*, vol. 6, no. 4, pp. 312–315, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405959519303455>
- [31] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [32] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [33] "Github copilot," <https://github.com/features/copilot>, accessed: 2025-03-10.
- [34] "Cvc5/src/theory/incomplete_id.h at main · cvc5/cvc5," https://github.com/cvc5/cvc5/blob/main/src/theory/incomplete_id.h.
- [35] I. Burnstein, *Practical software testing: a process-oriented approach*. Springer Science & Business Media, 2006.
- [36] B. Chen, A. A. Babikian, S. Feng, D. Varró, and G. Mussbacher, "Constraintsolver artifacts," Jun. 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.15679384>
- [37] L. Chen, Q. Guo, H. Jia, Z. Zeng, X. Wang, Y. Xu, J. Wu, Y. Wang, Q. Gao, J. Wang *et al.*, "A survey on evaluating large language models in code generation tasks," *arXiv preprint arXiv:2408.16498*, 2024.
- [38] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative AI: Assessing the role of LLMs," in *Generative AI for Effective Software Development*. Springer, 2024, pp. 129–148.
- [39] K. Ronanki, C. Berger, and J. Horkoff, "Investigating ChatGPT's potential to assist in requirements elicitation processes," in *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2023, pp. 354–361.
- [40] B. Görer and F. B. Aydemir, "Generating requirements elicitation interview scripts with large language models," in *RE 2023 Workshops (REW)*. IEEE, 2023, pp. 44–51.
- [41] K. Chen, Y. Yang, B. Chen, J. A. H. López, G. Mussbacher, and D. Varró, "Automated domain modeling with large language models: A comparative study," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 162–172.
- [42] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative ai in modeling tasks: an experience report with chatgpt and uml," *Software and Systems Modeling*, vol. 22, no. 3, pp. 781–793, 2023.
- [43] B. Chen, K. Chen, S. Hassani, Y. Yang, D. Amyot, L. Lessard, G. Mussbacher, M. Sabetzadeh, and D. Varró, "On the use of GPT-4 for creating goal models: An exploratory study," in *RE 2023 Workshops*. IEEE, 2023, pp. 262–271.
- [44] S. de Kinderen and K. Winter, "Towards taming large language models with prompt templates for legal GRL modeling," in *BPMDS 2024*. Springer, 2024, pp. 213–228.
- [45] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, "Fill in the blank: Context-aware automated text input generation for mobile GUI testing," in *ICSE 2023*. IEEE, 2023, pp. 1355–1367.
- [46] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *ICSE 2024*, 2024, pp. 1–13.
- [47] M. Sun, Y. Yang, Y. Wang, M. Wen, H. Jia, and Y. Zhou, "SMT solver validation empowered by large pre-trained language models," in *ASE 2023*. IEEE, 2023, pp. 1288–1300.
- [48] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, Z. Tian, Y. Huang, J. Hu, and Q. Wang, "Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model," in *ICSE 2024*, 2024, pp. 1–12.

- [49] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in neural information processing systems*, vol. 36, pp. 11 809–11 822, 2023.
- [50] J. Long, "Large language model guided tree-of-thought," *arXiv preprint arXiv:2305.08291*, 2023.
- [51] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.
- [52] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou *et al.*, "Metagtpt: Meta programming for multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, vol. 3, no. 4, p. 6, 2023.
- [53] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More agents is all you need," *arXiv preprint arXiv:2402.05120*, 2024.
- [54] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [55] D. Gao, Z. Li, X. Pan, W. Kuang, Z. Ma, B. Qian, F. Wei, W. Zhang, Y. Xie, D. Chen *et al.*, "Agentscope: A flexible yet robust multi-agent platform," *arXiv preprint arXiv:2402.14034*, 2024.
- [56] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision and the road ahead," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [57] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated unit test improvement using large language models at Meta," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 185–196.
- [58] Y. Yang, B. Chen, K. Chen, G. Mussbacher, and D. Varro, "Multi-step iterative automated domain modeling with large language models," in *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS)*, 2024, pp. 587–595.
- [59] A. Masoudifard, M. M. Sorond, M. Madadi, M. Sabokrou, and E. Habibi, "Leveraging graph-rag and prompt engineering to enhance llm-based automated requirement traceability and compliance checks," *arXiv preprint arXiv:2412.08593*, 2024.
- [60] A. A. Babikian, O. Semeráth, and D. Varró, "Concretization of Abstract Traffic Scene Specifications Using Metaheuristic Search," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 48–68, 2024.